

passgen

Руководство разработчика

Этот документ объясняет, как устроена программа passgen, какие в ней есть функции, за что каждая отвечает и как добавлять новые. Всё написано простым языком, без сложных терминов.

Программа состоит из одного файла index.html, в котором находятся и разметка (HTML), и оформление (CSS), и логика (JavaScript). Файл app.py запускает встроенный веб-сервер, который показывает этот файл.

1. Как устроены переменные

Что такое переменная

Переменная — это коробка с именем, в которую можно положить какое-то значение. Например, мы храним настройки в переменной `state`. Когда пользователь нажимает на галочку "Цифры", значение в `state` меняется с 0 на 1 или с 1 на 0.

Какие переменные есть

Переменная `state`

Хранит, какие наборы символов включены. У неё четыре свойства: `capitals` (1 или 0), `lowercase` (1 или 0), `digits` (1 или 0), `symbols` (1 или 0). Если значение 1 — набор включён, если 0 — выключен.

```
let state = {capitals:1, lowercase:1, digits:0, symbols:1};
```

Переменная `inv`

Хранит настройки инверсии (преобразования пароля после генерации). У неё два свойства: `cap` (0 или 1) — сделать первую букву заглавной, `dig` (0 или 1) — заменить последний символ на цифру.

```
let inv = {cap:0, dig:0};
```

Переменная `svcs`

Список сервисов, которые добавил пользователь. Например, если он ввёл "мой-сайт" и создал пароль, это название добавится в этот список и будет показываться в панели сервисов.

Константы — неизменяемые данные

CHK — наборы символов

Это массив (список) из четырёх элементов. Каждый элемент — это тоже маленький список из трёх частей: идентификатор (`id`), название для пользователя, и строка со всеми символами этого набора.

```
const CHK = [
  ['capitals', 'Заглавные', 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'],
  ['lowercase', 'Строчные', 'abcdefghijklmnopqrstuvwxyz'],
  ['digits', 'Цифры', '0123456789'],
  ['symbols', 'Спецсимволы', '!@#$%&*+=-_.'],
];
```

Чтобы добавить новый набор (например, скобки или иероглифы), нужно добавить сюда новую строку и добавить свойство в `state` со значением 0.

POPULAR — популярные сервисы

Список самых известных сервисов с логотипами. Если хотите добавить новый сервис в список, просто допишите новую строку в этот массив.

```
const POPULAR = [
  {n:'google', l:'g', c:'#4285f4'},
  {n:'github', l:'GH', c:'#333'},
  {n:'vk', l:'V', c:'#4a76a8'},
  {n:'yandex', l:'Ya', c:'#fc3f1d'},
];
```

2. Функции по очереди

init() — запуск программы

Эта функция вызывается автоматически в самом конце файла (последняя строка init()). Она делает три вещи:

- Рисует галочки наборов символов (renderChk)
- Рисует переключатели инверсии (renderInv)
- Загружает сохранённые настройки (loadState)

Также она прикрепляет обработчики событий: когда пользователь вводит текст в поле сида, убирается ошибка. Когда пользователь нажимает Enter в поле сида или сервиса — запускается генерация.

renderChk() — рисуем галочки

Эта функция полностью перерисовывает панель с галочками. Она берёт список СНК и для каждого набора создаёт элемент с кругляшкой и названием. Когда пользователь кликает по кругляшке, значение в state меняется на противоположное (0 -> 1, 1 -> 0), панель перерисовывается, и настройки сохраняются.

renderInv() — переключатели инверсии

Просто обновляет внешний вид двух переключателей "Первая заглавная" и "Последняя цифра". Если inv.cap = 1, у первого кружка появляется класс "on" (заполненный фиолетовый).

toggleInv(id) — переключение инверсии

Принимает id — строку "cap" или "dig". Меняет значение в inv на противоположное и обновляет оформление. Вызывается при клике по переключателю.

loadState() и saveState() — сохранение настроек

Программа запоминает настройки в хранилище браузера под именем "passgen_state". Данные не теряются, даже если закрыть и открыть программу. Хранится список сервисов, настройки наборов, инверсии и длина. Сид никогда не сохраняется.

Если вы добавите новую настройку, нужно добавить её и в loadState (чтобы восстанавливать), и в saveState (чтобы сохранять).

applyPreset() — предустановки

Когда пользователь выбирает вариант из выпадающего списка, эта функция устанавливает нужные галочки и длину:

- Минимальный: 8 символов, буквы + цифры
- Стандартный: 10 символов, всё кроме спецсимволов
- Максимальный: 16 символов, всё

adj(delta) — кнопки длины

Принимает число (1 или -1) и меняет значение длины. Если было 10 и нажать +1, станет 11. Значение не может стать меньше 1 или больше 99.

3. Функции сервисов

toggleSvcPanel() — панель сервисов

Когда пользователь нажимает "список" рядом с полем сервиса, появляется панелька со списком. Эта функция просто переключает классы open на панели и на стрелке. CSS делает анимацию раскрытия.

renderSvcChips() — рисуем список сервисов

Эта функция полностью пересобирает сетку с чипами (маленькие плашки с названиями). Она берёт популярные сервисы из POPULAR и добавленные пользователем из svcs. Популярные выводятся первыми, потом пользовательские.

chip(item, hasLogo, deletable) — создание одного чипа

Эта функция создаёт один маленький прямоугольник с названием сервиса. Она принимает три параметра:

- item — что показать (строка или объект с n, l, c)
- hasLogo — есть ли логотип (true/ложь)
- deletable — можно ли удалить по правому клику (правда/ложь)

Если есть логотип, рисуется цветной кружок с буквой. Если логотипа нет — серый кружок с "?". При клике по чипу название попадает в поле ввода. При правом клике по пользовательскому сервису он удаляется.

exportData() и **importData(e)** — выгрузка и загрузка

Экспорт создаёт JSON-файл со всеми настройками, а импорт загружает их из файла обратно. Сид никогда не попадает в файл экспорта — это сделано намеренно, чтобы не случайно его кому-то передать.

Импорт проверяет, что в файле есть нужные поля (svcs, state, inv, len) и восстанавливает только их. Если файл испорчен, покажет ошибку.

toggleAdvanced() — расширенные настройки

Раскрывает или сворачивает блок с переключателями "Первая заглавная" и "Последняя цифра". Когда блок раскрыт, стрелка поворачивается на 90 градусов.

4. Функции генерации

gen() — главная функция создания пароля

Эта функция запускается, когда пользователь нажимает кнопку "Создать". Она делает вот что по шагам:

- Берёт значение из поля "Сид"
- Берёт значение из поля "Сервис" (если пусто — ставит "(empty)")
- Берёт длину из поля длины
- Проверяет, заполнен ли сид. Если нет — показывает ошибку.
- Проверяет, хотя бы один набор символов включён.
- Собирает алфавит из включённых наборов
- Вызывает hmacGen() для получения пароля
- Если включена "Первая заглавная" — делает первый символ заглавным
- Если включена "Последняя цифра" — через HMAC считает цифру и заменяет последний символ
- Вставляет пароль в поле, копирует в буфер обмена
- Если сервис новый — добавляет его в список
- Сохраняет настройки и обновляет полоски надёжности

hmacGen(seed, svc, len, alpha) — сердце программы

Эта функция создаёт пароль с помощью алгоритма HMAC-SHA256. Вот как это работает:

- Она берёт сид как ключ
- Берёт название сервиса и длину как сообщение
- Получает HMAC-подпись (256 бит)
- Каждый байт подписи превращается в символ из алфавита
- Если нужно больше символов, чем в одной подписи — увеличивает счётчик и считает новую

Важный момент: чтобы распределение символов было равномерным, используется "rejection sampling" (отбрасывание). Если байт больше или равен 256 - (256 % длина алфавита), он пропускается. Это гарантирует, что все символы будут выпадать с одинаковой вероятностью.

genLogin() — генерация логина

Создаёт короткий из 6 символов логин на основе SHA-256. Используется алфавит без путающихся символов: нет l, o, 0 и 1. Считывается по байтам из хэша от сиду + "login" + сервис.

Логин предназначен для того, чтобы его можно было запомнить, а не просто скопировать.

updateStrength(len, sets) — оценка надёжности

Эта функция оценивает, насколько пароль надёжен. Она рисует полоски и текст под полем пароля.

- Слабый: меньше 10 символов или меньше 3 наборов. Цвет розовый.
- Средний: от 10 символов и 3 набора. Цвет малиновый.
- Сильный: от 14 символов и 4 набора. Цвет сиреневый.

Каждому уровню соответствует текст с рекомендацией.

cpy() — копирование

Берёт пароль из поля и копирует в буфер обмена. Если современный метод не сработает, использует запасной (выделение текста и exesCommand). Показывает надпись "Скопировано!" на 1.5 секунды.

5. app.py — веб-сервер

Файл `app.py` запускает простой HTTP-сервер, который показывает файл `index.html`. Он написан на Python и использует только встроенные библиотеки. Никаких дополнительных пакетов устанавливать не нужно.

Как это работает

- Сервер запускается на порту 8765
- Он просто раздаёт файлы из своей папки
- Логи запросов отключены (чтобы не мусорить в терминал)
- Если порт 8765 занят, автоматически ищет свободный
- Работает в фоновом потоке
- Остановка по `Ctrl+C`

Что можно изменить

- Поменять `PORT` на другой
- Включить логи: убрать переопределение `log_message`
- Добавить обработку других URL, например `/api/save`
- Запускать на `0.0.0.0`, чтобы было видно по сети

6. Как добавить новую функцию

Простой пример: добавить новый набор символов

Допустим, хотите добавить набор "Скобки" с символами `()[]{}.` Нужно сделать три вещи:

Шаг 1. Добавить в СНК

```
const СНК = [  
  ['capitals', 'Заглавные', 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'],  
  ['lowercase', 'Строчные', 'abcdefghijklmnopqrstuvwxyz'],  
  ['digits', 'Цифры', '0123456789'],  
  ['symbols', 'Спецсимволы', '!@#%&*+=-~.'],  
  ['brackets', 'Скобки', '()[]{}'],  
];
```

Шаг 2. Добавить в state

```
let state = {capitals:1, lowercase:1, digits:0, symbols:1, brackets:0};
```

Шаг 3. Добавить в пресеты (если нужно)

```
case 'max': state={capitals:1, lowercase:1, digits:1, symbols:1, brackets:1}; length.value=16; break;
```

Готово! Программа автоматически подхватит новый набор, потому что `renderChk()` проходит по всему СНК, а `hmacGen()` собирает алфавит из включённых наборов.

Более сложный пример: добавить новую кнопку

Допустим, хотите добавить кнопку "Очистить всё".

Шаг 1. Добавить в HTML

```
<button class="btn btn-secondary"  
  onclick="clearAll()">Очистить всё</button>
```

Шаг 2. Написать функцию

```
function clearAll(){  
  seedInput.value = "";  
  serviceInput.value = "";  
  pwdField.value = "";  
  loginField.value = "";  
  seedInput.classList.remove('error');  
  seedErr.textContent = "";  
}
```

Как изменить алгоритм HMAC

В функции `hmacGen()` можно:

- Заменить хэш на SHA-512: поменять строку в `importKey`
- Изменить формат сообщения: добавить данные в `msgBase`
- Изменить счётчик: вместо байта использовать 4 байта

Как добавить новый цвет в дизайн

Все цвета в одном месте — в `:root` в начале CSS. Просто добавьте новую переменную, например:

```
:root {  
  --orange: #f97316;  
}
```

Потом используйте в стилях: `color: var(--orange).`

7. Структура проекта

Все файлы проекта:

```
pract/  
index.html    # основное приложение  
app.py        # HTTP-сервер  
README.md     # инструкция по установке  
.gitignore  
manual.pdf    # пользовательское руководство  
dev_manual.pdf # этот документ
```

Запуск

Для запуска программы выполните:

```
python3 app.py # Linux  
python app.py  # Windows
```